

Annotated Bibliography

Daizy Buluma

Exploring k-Nearest Neighbors in Recommendation Systems

1. Introduction

Today's digital world is overflowing with information, making it nearly impossible for individuals to manually sift through the massive number of available options. As a result, recommender systems have become essential components of platforms such as Netflix, Amazon, Spotify, and TikTok. Their primary goal is to filter vast collections of items such as movies, products, songs, or videos, and present users with options they are most likely to enjoy. These systems work by analyzing users' preferences, past behaviors, and item characteristics through interaction data such as impressions, clicks, ratings, likes, and purchases (Ghosh, 2024).

This paper focuses on the k-nearest neighbors (kNN) algorithm within the context of recommendation systems. Although kNN is one of the more straightforward algorithms in machine learning, it plays a foundational role in collaborative filtering methods. kNN identifies users or items that are most similar in behavior or preferences and bases predictions on these neighbors. Its simplicity and intuitive mechanics make it especially suitable for educational purposes and small scale recommendation tasks.

Before exploring the details of kNN, this paper outlines the broader landscape of recommendation systems to provide a foundation for understanding how kNN fits into the larger family of recommendation systems. The discussion then introduces and contrasts a different recommendation approach, Singular Value Decomposition (SVD) to highlight the strengths and weaknesses of kNN in practice.

Finally, the second half of the paper applies kNN to a real dataset to build and evaluate a simple interactive recommender system so as demonstrate how the method works in practice gauge its performance.

2. Overview of Recommender Systems

Recommendation systems work by estimating how much a user is likely to enjoy an item they have not yet interacted with. They do this by analyzing User behavior, such as past ratings or interactions and Item features, such as genre, creator, or textual descriptions. The goal is to estimate a preference score, $r_{u,i}$, which measures how likely user u is to enjoy item i . Recommendations are then generated by selecting the items with the highest scores.

To understand how k-nearest neighbors (kNN) fits into this broader landscape, it is useful to review two major families of recommendation approaches: **content-based approach** and **collaborative filtering** (There are hybrid recommenders too).

2.1 Content-Based Filtering

Content-based recommender systems generate suggestions by comparing the attributes of items a user previously liked to those of new items. These attributes might include genres, keywords, textual descriptions, or metadata.

For example:

- If a user watches many nature documentaries, the system recommends similar documentaries.
- If a shopper frequently purchases skincare products, the system recommends related cosmetic items.

The underlying assumption is that user preferences are largely consistent, and similar items will yield similar satisfaction (Ghosh, 2024). These models rely heavily on item features, making them effective in domains where rich metadata is available.

2.2 Collaborative Filtering

Collaborative filtering takes a different approach by focusing on patterns of behavior across users rather than item attributes. The key assumption is that: “Users who behaved similarly in the past will behave similarly in the future.” (Ricci et al., 2022) This method relies solely on user–item interactions such as ratings, likes, or clicks. Collaborative filtering is especially powerful in situations where item metadata is limited or unstructured. There are two main subtypes:

a) Memory-Based Methods

Memory-based methods make predictions directly from observed ratings by identifying similarities among users or items. kNN is the most common example.

- **User-based kNN:** Finds users who rate items similarly.
- **Item-based kNN:** Finds items that receive similar ratings from users.

These systems are intuitive, interpretable, and work well for small to medium-sized datasets.

b) Model-Based Methods

Model-based approaches use machine learning techniques to uncover latent patterns in the rating matrix. Examples include: **Matrix factorization and SVD and Neural collaborative filtering**. These methods compress users and items into low-dimensional representations that capture hidden “taste” factors (Aggarwal, 2016). They tend to outperform memory-based methods on large, sparse datasets but are less transparent to users and developers.

Notably, many hybrid recommenders combine kNN with model-based techniques, such as SVD, to balance simplicity and predictive power (Hssina et al., 2021).

3. K-Nearest Neighbors in Recommendation Systems

This section provides a deeper exposition of how kNN works, beginning with motivation and intuition, then moving into supervised-learning kNN, and finally examining how the algorithm is adapted for recommendation systems. A worked toy example concludes the section to illustrate the computations concretely.

3.1 Motivation: Why Neighbors Matter in Recommendations

The intuition behind neighborhood-based methods is straightforward. As we’ve established in the section above, users and items rarely exist in isolation, but instead form clusters of shared preferences. By identifying these clusters, recommender systems can exploit the collective wisdom of similar users to generate predictions. For example, if Alice and Bob consistently rate the same movies in a similar way, Bob’s positive rating of a film that Alice has not yet watched becomes a strong signal that Alice may also enjoy it. This neighbor-driven reasoning allows kNN to provide recommendations without complex modeling, relying instead on the natural tendency of like-minded users to overlap in their choices (Nikolakopoulos et al., 2021).

3.2 How kNN Works in Supervised Learning

Before adapting k-nearest neighbors (kNN) to recommender systems, it helps to first understand how the algorithm works in its original setting. kNN is one of the simplest prediction methods because it does **not** build a model in the usual sense (hence why is referred to as lazy). There are no parameters to estimate and no equations to fit. Instead, kNN makes predictions by **comparing new data points to existing examples**.

In classification tasks, the new example is assigned to the majority class among its closest neighbors. In regression, the prediction is the average of their values. The choice of how many neighbors to consult, k , is crucial. A very small k can make the algorithm sensitive to noise, while a larger k smooths predictions but may dilute the influence of the most relevant points.

Classic kNN treats all k neighbors equally, but weighted kNN assigns larger influence to closer points. This reduces the impact of outliers and improves performance in many real-world settings. These ideas carry over directly into recommendation systems, but with one major complication: *missing data*. Most users rate only a tiny fraction of items, which forces kNN to compute distances and similarities on overlapping subsets.

3.3 How kNN Works in Recommender Systems

a) The User–Item Matrix

Recommendation systems typically begin with a sparse matrix of size (number of users) $\times$ (number of items). An illustrative example is:

User / Item	A	B	C	D
U1	5	4	?	3
U2	4	5	2	?
U3	?	4	4	3
U4	2	?	5	4

Most cells are empty, reflecting the high sparsity typical in modern datasets:

kNN must therefore compute similarities using only the items (or users) that overlap. This reliance on shared ratings is both a strength because it uses actual behavioral similarity and a weakness because sparsity may cause the similarities to be unreliable, as noted by Ricci et al. (2022) and Kramer (2013).

b) User-Based vs. Item-Based kNN

User-based kNN identifies the k users whose rating patterns most closely resemble the target user.

Example: To predict how User 1 will rate Item C, we find other users who have rated Item C and whose rating histories correlate with User 1's past ratings.

User-based kNN typically works best when:

- Users have rated many of the same items.
- The dataset has more items than users (common in early recommendation systems, less common today).
- User tastes form tight clusters.

Item-based kNN, made famous by Amazon's item-to-item system (Linden et al., 2003), instead finds items that consistently receive similar ratings across users.

Example: If Item C is rated similarly to Item A across many users, then a user who liked Item A is predicted to like Item C as well.

Item-based kNN tends to be more stable over time because item relationships remain consistent even as new users join and others leave. For this reason, modern commercial recommenders tend to use item-based kNN over user-based (Ricci et al., 2022).

c) Similarity Metrics for Collaborative Filtering

In recommendation settings, not all distance metrics are equally effective. The most common are:

Cosine Similarity

Cosine similarity focuses on the angle between rating vectors rather than their magnitude:

$$\text{sim}(i, j) = \frac{\sum_u r_{u,i} r_{u,j}}{\sqrt{\sum_u r_{u,i}^2} \sqrt{\sum_u r_{u,j}^2}}$$

It is widely used because it remains stable even when users use different rating scales (Aggarwal, 2016).

Pearson Correlation

Pearson correlation adjusts for user biases:

$$\text{sim}(i, j) = \frac{\sum_u (r_{u,i} - \bar{r}_u)(r_{u,j} - \bar{r}_u)}{\sqrt{\sum_u (r_{u,i} - \bar{r}_u)^2} \sqrt{\sum_u (r_{u,j} - \bar{r}_u)^2}}$$

This metric reduces problems with users who rate too generously or too strictly.

Adjusted Cosine

Used mostly in item-based kNN, this version subtracts user means before computing cosine:

$$\text{sim}(i, j) = \frac{\sum_u (r_{u,i} - \bar{r}_u)(r_{u,j} - \bar{r}_u)}{\sqrt{\sum_u (r_{u,i} - \bar{r}_u)^2} \sqrt{\sum_u (r_{u,j} - \bar{r}_u)^2}}$$

It combines strengths of cosine and Pearson.

d) Prediction Formula

Once similarities have been computed, prediction is done using weighted averages.

User-Based kNN

For predicting user (u)'s rating for item (i):

$$\hat{r}_{u,i} = \frac{\sum_{v \in N_k(u)} \text{sim}(u, v) r_{v,i}}{\sum_{v \in N_k(u)} |\text{sim}(u, v)|}$$

Item-Based kNN

For predicting user (u)'s rating for item (i):

$$\hat{r}_{u,i} = \frac{\sum_{j \in N_k(i)} \text{sim}(i, j) r_{u,j}}{\sum_{j \in N_k(i)} |\text{sim}(i, j)|}$$

Both formulas emphasize closer neighbors more heavily.

c) Toy Example

We finish with toy examples adapted from Agrawal's book to emphasize idea:

Ratings:

User	A	B	C
U1	5	4	?
U2	4	4	2
U3	1	1	1

Goal: Predict U1's rating for Item C.

Step 1: Compute similarities

Compare Item A vs Item C using cosine similarity:

$$A = (4, 1) \quad C = (2, 1)$$

$$\text{sim}(A, C) \approx 0.98$$

Step 2: Weighted rating

U1 rated:

- $A = 5$
- $B = 4$

If A is most similar to C: $\hat{r}_{U1,C} \approx 5 \times 0.98 = 4.9$

Thus U1 is very likely to enjoy Item C.

3.4 Strengths and Weaknesses of kNN in Recommendation Systems

- Easy to Understand and Explain
- No Training Phase
- Works Well for Small or Dense Dataset

4. Comparison to a Model-Based Method (SVD)

- What SVD does (very high level)
- How it differs from kNN
- Strengths and weaknesses of both
- Maybe Table comparison ?
- When to choose kNN vs SVD
- Why SVD is dominant in large-scale systems

5. Application: Building a Simple Recommendation Engine with kNN

5.1 Dataset Description + Data Wrangling

5.2 Implementing kNN + Model Evaluation

5.3 Results + Interpretation

6. Reflection & Limitations

7. Conclusion

8. References

Ricci, F., Rokach, L., & Shapira, B. (Eds.). (2022). *Recommender systems handbook* (3rd ed.). Springer. <https://doi.org/10.1007/978-1-0716-2197-4>

Aggarwal, C. C. (2016). *Recommender systems: The textbook*. Springer. <https://doi.org/10.1007/978-3-319-29659-3>

Ghosh, A. (2024, May 21). *Recommendation system: A complete guide*. LearnOpenCV. <https://learnopencv.com/recommendation-system/#aioseo-types-of-recommendation-system>

Kramer, O. (2013). *Dimensionality reduction with evolutionary algorithms*. Springer. <https://doi.org/10.1007/978-3-642-38652-7>

Hssina, B., Grotta, A., & Erritali, M. (2021). Recommendation system using the k-nearest neighbors and singular value decomposition algorithms. *International Journal of Electrical and Computer Engineering*, 11(6), 5541–5548. <https://doi.org/10.11591/ijece.v11i6.pp5541-5548>

Nikolakopoulos, A. N., Ning, X., Desrosiers, C., & Karypis, G. (2021). Trust your neighbors: A comprehensive survey of neighborhood-based methods for recommender systems. *arXiv preprint arXiv:2109.04584*. <https://arxiv.org/abs/2109.04584>

9. Appendix